Programming with Python

Group Assignment

Hand in date: **Week 3**

Handout date: **3 November 2024**

Group number: **Group 50**

Lecturer: **Muhammad Huzaifah Ismail**

Group Members

Name	TP Number
HEIN HTET	TP081818
MOHAMMAD ABU NUR TASFI	TP080772
AUNG KAUNG KHANT	TP082184
MOE LWIN PAING HTOO	TP081628

Table of Contents

Introduction:

For this assignment, we designed an extensive Library Management System for Brickfields Kuala Lumpur Community Library. This system is based on various accounts that would be created for its users: a super admin, system administrator, librarian, and library member. Each user plays a certain role in fulfilling the functionality needed to make sure that the library functions well. Administrator and super admin accounts maintain the security of the system and handle data of both librarians and members for correct record keeping and access into the accounts. librarians can add and manage a catalog of books, besides handling the loan process for the library members. The members will be able to log in to see their current loans, search for books, see due dates, and update their profiles. The entire management system is developed in Python. The system uses text files as its storage means in such a way, that it automates all library tasks that require manual input.

Assumption:

Without a digital Library Management System, the Brickfield Community Library has to resort to traditional, ‘manual’ procedures like book cards, physical logs, and paper records of daily operations. Once standard, this approach, however, now has severe limitations, being quite challenging in larger libraries with large collections and high transaction volumes. Librarians need to spend a lot of time recording who borrows and returns books, how many books, and where they are and when. By doing so, it not only increases workload but also is more likely to have human error. Inconsistent policies, inefficient workflows, and just plain frustration for library users all could result in lost records, misfiled book cards, and an error in calculating fines (Drabenstott et al., 1989).

For the library members, the limitations of a manual system directly have to do with their experience: locating a particular book will involve searching through physical catalogs most of the time or even seeking assistance from staff. The members may have very limited information about their loan status or fines without the assistance of the staff and are dependent on the librarian for the basic details of their account. A manual system fails to meet the expectations of the users. As such, patrons may regard any visits as time-wasting or even avoidant if the services are not user-friendly (Kong & Xia, 2017).

Furthermore, the use of book cards and paper records for manual keeping, though effective in historical times, turns out increasingly inefficient in handling growing collections. For example, librarians have to update the loan records by hand, and due dates, and calculate the overdue fees for every transaction. The possibility of an error is high since the small mistakes accumulate and

may disrupt the accuracy of the library's records. The management of overdue fees is particularly inconvenient since tracking is not automated and a librarian relies on calculations that might be prone to mistakes (McGee, 1973). The more the collection grows, the worse this problem is because it gets increasingly difficult to keep accurate, up-to-date records and the time and effort required to do this manually can become almost impossible to manage.

The digital library system helps save manual efforts by automating most of the basic functions and further improving accuracy. The system allows librarians to input all the information about books and transactions at once; after which they become instantly searchable and available. This also serves to improve catalog management, giving more reliable book tracking for which the librarians are freed to assist patrons instead of spending time in administrative work.

The members in this case can search for a catalog, confirm the availability of a book, and due dates, and renew the loans online, further reducing reliance on library staff. The automatic tracking and calculation of overdue charges apply policy consistently while minimizing errors prevalent in manual systems (Putra & Labasariyani, 2022). It is also helpful for the library to monitor and analyze the trends of the borrowings, which enables them to understand the popular and unpopular items in the collection. This can facilitate the library's fine-tuning of its collections to suit the prevailing needs of users on the ground and hence make the resources applicable in society (A & Rathi, 2023). Besides this, record losses or damages are highly minimized with digital data storage; thus, information is both secure and organized.

Finally, although manual systems like book cards were used to meet library requirements earlier, they do not meet modern criteria for efficiency, accuracy, and user contentment. A Digital Library Management System for the Brickfields Kuala Lumpur Community Library will greatly enhance its operation by automating cataloging, user account maintenance, and overdue keeping. It lessens manual errors, increases user independence, and lands the library as a progressive community-based organization.

Program Design

 # Set the global variables

```
SET user_data = {'Member': [], 'Librarian': [], 'Admin': []}
SET user_type = None
SET user_database = 'account.txt'
SET loans_file = 'loanedmembers.txt'
SET books_file = 'books.txt'
SET current_time = datetime.now()
```

Login Part

 # User Credentials verification

```
FUNCTION login_users(data):
  CALL Generate_title: "Welcome to Brickfields Kuala Lumpur Community Library"

   WHILE True:
     SET username = INPUT("Enter your email address")
     SET password = INPUT("Enter your password")

      FOR each category in data:
        FOR each user in category:
          IF user email matches username AND user password matches password:
            PRINT "Login successful! Welcome [username], Role: [role]"
            RETURN user
          ENDIF
        ENDFOR
      ENDFOR

      PRINT "Invalid email or password, try again"
  ENDWHILE
```

```
# Choose login interface for different types of accounts
FUNCTION select_interface(user):
  SET interface TO user["role"]

   IF interface == "admin":
      CALL sys_admin(interface)
   ELSE IF interface == "super_admin":
      CALL super_admin(interface)
   ELSE IF interface == "member":
      CALL libmeminterface(user)
   ELSE IF interface == "librarian":
      CALL librarian_interface()
   ENDIF

# Main interface for login function
FUNCTION main():
  CALL load_data() and ASSIGN TO account_data

    WHILE True:
     CALL login_users(account_data) and ASSIGN TO user

       IF user EXISTS:
        CALL select_interface(user)
       ENDIF
    ENDWHILE
```

Admin Part

 # Admin Interface

```
START
   FUNCTION sys_admin(role = 'admin'):
      SET picked_num = options(role)

      IF picked_num == 1:
         CALL member_manage(role)
      ENDIF
      IF picked_num == 2:
         CALL librarian_manage(role)
      ENDIF
      IF picked_num == 3:
         PRINT '[*] You have successfully logged out!'
         CALL main()
      ENDIF
END
```

 # Super_admin Interface

```
START
   FUNCTION super_admin(role = 'super_admin'):

      SET picked_num = options(role)
      IF picked_num == 1:
         CALL member_manage(role)
      ENDIF
      IF picked_num == 2:
         CALL librarian_manage(role)
      ENDIF
      IF picked_num == 3:
         CALL admin_manage()
      ENDIF
      IF picked_num == 4:
         PRINT '[*] You have successfully logged out!'
         CALL main()
      ENDIF
END
```

Admin main Pannel Options

```
START
   FUNCTION  options(role):
      CALL generate_title('Welcome to Admin Panel')
      SET option_list = ['Member Account Management', 'Librarian Account Management',
'Logout']

      IF role == 'super_admin':
         option_list.insert(2, 'Admin Account Management')
      CALL generate_options(option_list)
      ENDIF

      WHILE True:
         SET picked_num = INPUT(f'[*] Enter the number to manage the users: ')
         TRY:
            SET input_value = int(picked_num)
            IF NOT 0 < input_value <= len(option_list):
               RAISE ValueError
            ENDIF
            RETURN input_value

         CATCH ValueError:
            PRINT '[ERROR] Invalid input! Please enter again.'
            CONTINUE
      ENDWHILE
END
```

Commands Options

```
START
   FUNCTION cmd_option(user_type):
      SET options = f'"
      Available commands:
      - add: Add a new {user_type}
      - view: View {user_type} details
      - search: Search {user_type} information
      - edit: Edit {user_type} information
      - delete: delete a {user_type}
      - options: Show the available commnads
      - exit: Back to Main Menu
```

```
    '''
    PRINT options
END
```

Save Data

```
START
  FUNCTION save_data(file_name, data):
    OPENFILE file_name FOR WRITE AS f:
      json.dump(data, f, indent=4)
END
```

Load Data

```
START
  FUNCTION load_data():
    GLOBAL user_data
    TRY:
      IF path of file exists:
        OPENFILE user_database FOR READ AS f:
          user_data = json.load(f)
          RETURN user_data
      ELSE:
        SET user_data = {'Member': [], 'Librarian': [], 'Admin': []}
      ENDIF
    CATCH json.JSONDecodeError:
      SET user_data = {'Member': [], 'Librarian': [], 'Admin': []}
END
```

Generate Title

```
START
  FUNCTION generate_title(title):
    PRINT ''
    PRINT f'{title.title()}'
    PRINT f'{"-" * 15}'
END
```

Generate Option List

```
START
  FUNCTION generate_options(option_list):
```

```
    SET number = 1
    FOR i in option_list:
      PRINT f'{number}. {i.title()}'
      SET number += 1
    ENDFOR
    PRINT ''
END
```

Generate id for users

```
START
  FUNCTION generate_id():
    load_data()
    GLOBAL user_type
    SET member_type = user_type.title()

    IF NOT user_data[member_type]:
      RETURN 101
    ENDIF

    SET max_id = max(int(user['id']) FOR user in user_data[member_type])
    RETURN max_id + 1
END
```

Interface for member management

```
START
  FUNCTION member_manage(role):
    PRINT ''
    CALL generate_title('Member Account Management')
    PRINT '[*] Use [options] to view the available commands.\n'

    WHILE True:
      GLOBAL user_type
      SET user_type = 'member'
      SET command = INPUT(f'Admin~# ').strip()
      IF command == 'add':
        CALL add_data()
      ELIF command == 'view':
        CALL view_data()
      ELIF command == 'search':
        CALL search_data()
```

```
        ELIF command == 'edit':
            CALL edit_data()
        ELIF command == 'delete':
            CALL delete_data()
        ELIF command == 'options':
            CALL cmd_option(user_type)
        ELIF command == 'exit':
            PRINT ''
            IF role == 'admin':
                CALL sys_admin()
            ENDIF
            IF role == 'super_admin':
                CALL super_admin()
            ENDIF
        ELSE:
            PRINT f"[ERROR] Unknown command. Type 'options' for a list of available
commands."
        ENDIF
    ENDWHILE
END
```

Interface for librarian management

```
START
  FUNCTION librarian_manage(role): #Librarian Management Panel
    PRINT ''
    CALL generate_title('Librarian Account Management')
    PRINT "[*] Use [options] to view the available commands.\n"

    WHILE True:
        GLOBAL user_type
        SET user_type = 'librarian'
        SET command = INPUT(f'Admin~# ').strip()

        IF command == 'add':
            CALL add_data()
        ELIF command == 'view':
            CALL view_data()
        ELIF command == 'search':
            CALL search_data()
        ELIF command == 'edit':
```

```
            CALL edit_data()
        ELIF command == 'delete':
            CALL delete_data()
        ELIF command == 'options':
            CALL cmd_option(user_type)
        ELIF command == 'exit':
            PRINT ''
            IF role == 'admin':
                CALL sys_admin()
            ENDIF
            IF role == 'super_admin':
                CALL super_admin()
            ENDIF
        ELSE:
            PRINT f"[ERROR] Unknown command. Type 'options' for a list of available
commands."
        ENDIF
    ENDWHILE
END
```

Interface for admin management

```
START
  FUNCTION admin_manage():
    PRINT ''
    CALL generate_title('Admin Account Management')
    PRINT '[*] Use [options] to view the available commands.\n'

    WHILE True:
      GLOBAL user_type
      SET user_type = 'admin'
      SET command = INPUT(f'Admin~# ').strip()
      IF command == 'add':
        CALL add_data()
      ELIF command == 'view':
        CALL view_data()
      ELIF command == 'search':
        CALL search_data()
      ELIF command == 'edit':
        CALL edit_data()
      ELIF command == 'delete':
```

```
            CALL delete_data()
        ELIF command == 'options':
            CALL cmd_option(user_type)
        ELIF command == 'exit':
            PRINT "
            CALL super_admin()
        ELSE:
            PRINT f"[ERROR] Unknown command. Type 'options' for a list of available
commands."
        ENDIF
    ENDWHILE
END
```

Function for role_validation

```
START
  FUNCTION role_validation():
    GLOBAL user_type
    SET roles = ['member', 'librarian', 'admin']
    WHILE True:
      SET role = INPUT('Enter the role: ').lower().strip()

      IF user_type.lower() == 'member' and role == roles[0]:
            RETURN role
      ELIF user_type.lower() == 'librarian' and role == roles[1]:
            RETURN role
      ELIF user_type.lower() == 'admin' and role == roles[2]:
            RETURN role
      ELSE:
        PRINT f'[ERROR] Invalid role! Only "{user_type.lower()}" role is accepted.'
        CONTINUE
      ENDIF
    ENDWHILE
END
```

Validate the user information

```
START
  GLOBAL user_type
  SET member_type = user_type.title()
  SET patterns = {
      'full_name': re.compile(r'^[a-zA-Z0-9][\w\s.-_]{0,18}$'),
```

```
        'email': re.compile(r'^[\w\.-]+@[\w\.-]+\.\w+$'),
        'username': re.compile(r'^[a-zA-Z0-9][\w.-]{1,13}$'),
        'password': re.compile(r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&]{1,5})[A-Za-
z\d@$!%*?&]{8,}$')
    }
  SET pattern = patterns.get(pattern_key)

  IF NOT pattern:
    RAISE ValueError('Invalid Pattern Key!')
  ENDIF

  WHILE True:
    INPUT input_value
    SET match = pattern.fullmatch(input_value)

    IF match:
      IF pattern_key == 'email':
        IF ANY(user['email'] == input_value FOR user in user_data[member_type]):
          PRINT f'[ERROR] This email is already registered. Please use a different email.'
          CONTINUE
        ENDIF
      ENDIF

      IF pattern_key == 'full_name':
        IF ANY(user['full_name'] == input_value FOR user in user_data[member_type]):
          PRINT f'[ERROR] This name is already used. Please choose a different first name.'
          CONTINUE
        ENDIF
      ENDIF
      IF pattern_key == 'username':
        IF ANY(user['username'] == input_value FOR user in user_data[member_type]):
          print f'[ERROR] This username is already used. Please choose a different username.'
          CONTINUE
        ENDIF
      ENDIF
      RETURN input_value
    ENDIF
    ELSE:
      SET error_messages = {
        'full_name': 'Only alphabets, digits, and hyphens are allowed! Maximum length is 20
```

```
characters.',
            'email': 'Invalid email address format! Example: example@domain.com',
            'username': 'Invalid Username! Maximum length is 15 characters. Example:
user_name123',
            'password': 'The password must be at least 8 characters long and include:\n- At least
one uppercase letter\n- At least one lowercase letter\n- At least one number\n- At least one
special character (@, $, !, %, *, ?, &)'
        }
        PRINT f'[ERROR] {error_messages.get(pattern_key, "Invalid input.")}'
  ENDWHILE
END
```

Function to display data in a specific format

```
START
  FUNCTION show_data(data_list, counter):
    PRINT "
    PRINT f"{'Id':<10} {'Full_name':<20} {'Email':<25} "
        f"{'Username':<20} {'Password':<22} {'Role':<14} {'Registration_date':<20}"

    FOR data in data_list:
      PRINT f"{data.get("id"):<10} {data.get("full_name"):<20} {data.get("email"):<25} "
          f"{data.get("username"):<20} {data.get("password"):<22} {data.get("role"):<14}
{data.get("registration_date"):<20}"
    ENDFOR
    PRINT "

    IF counter == 0:
      PRINT f'\n[*] No data found!'
    ENDIF
    IF counter > 0:
      PRINT f'[*] {counter} user{"" IF counter == 1 OR 0 ELSE "s"} found!'
    ENDIF
    PRINT "
END
```

Function to add user

```
START
  FUNCTION add_data(): #To add user data into the database
    TRY:
      CALL load_data()
```

```
        GLOBAL user_type
        SET member_type = user_type.title()

        SET data = {
            'id': generate_id(),
            'full_name': CALL validate_data('full_name'),
            'email': CALL validate_data('email'),
            'username': CALL validate_data('username'),
            'password': CALL validate_data('password'),
            'role': CALL role_validation(),
            'registration_date': datetime.now().strftime('%d-%B-%Y')
          }

        user_data[member_type].append(data)
        CALL save_data(user_database, user_data)
        PRINT f'[*] The {member_type} has been created successfully.\n'
    CATCH Exception as e:
        PRINT f'[ERROR] An error occurred while adding the data: {e}'
END
```

Function to view existing users

```
START
  FUNCTION view_data():
    CALL load_data()
    GLOBAL user_type
    SET member_type = user_type.title()
    CALL show_data(user_data[member_type], len(user_data[member_type]))
END
```

Function to search users

```
START
  FUNCTION search_data():
    CALL load_data()
    GLOBAL user_type
    SET member_type = user_type.title()

    WHILE True:
      SET option_list = ['id', 'full_name', 'email', 'username', 'exit']
      SET search_key = INPUT(f'\033[93m\n[*]\033[0m Options: (id, full_name, email,
username, \'exit\' to quit)\n'
```

```
                f'Which field would you like to search?: ').lower()

        IF search_key == 'exit':
            PRINT f'[*] Exiting the searching process.\n'
            BREAK

        TRY:
            IF NOT search_key in option_list:
                RAISE ValueError
            ENDIF

            SET counter = 0
            SET search_value = INPUT('Enter the keyword to search: ')
            SET data_list = []

            FOR data in user_data[member_type]:
                SET match_data = re.search(search_value, str(data[search_key]),
flags=re.IGNORECASE)
            ENDFOR

                IF match_data:
                    data_list.append(data)
                    counter += 1
                ENDIF
            CALL show_data(data_list, counter)
        CATCH ValueError:
            PRINT f'[ERROR] Invalid option! Please enter the valid option.'
    ENDWHILE
END
```

Function to edit users' information

```
START
  FUNCTION edit_data():
    CALL view_data()
    GLOBAL user_type
    SET member_type = user_type.title()

    WHILE True:
        SET id = INPUT(f'[*] Enter the {user_type}\'s ID you want to edit (or type \'exit\' to
quit): ').lower()
```

```
        IF id == 'exit':
            PRINT f'[*] Exiting the editing process.\n'
            BREAK
        ENDIF

        TRY:
            SET max_id = max(int(data['id']) FOR data in user_data[member_type]) + 1
            SET valid_id = int(id)

            IF NOT 100 < valid_id < max_id:
                RAISE ValueError("ID out of acceptable range.")
            ENDIF

            FOR data in user_data[member_type]:
                IF valid_id == data['id']:
                    WHILE True:
                        SET choice_to_edit = input(
                            f' Options: (full_name, email, username, password)\n'
                            f'[*] Which field would you like to update?: ').lower()
                        SET option_list = ['full_name', 'email', 'username', 'password',]
                        IF choice_to_edit in option_list:
                            SET edited_data = CALL validate_data(choice_to_edit)
                            SET data[choice_to_edit] = edited_data
                            CALL save_data(user_database, user_data)
                            PRINT f[*] The {member_type} has been edited successfully.'

                            WHILE True:
                                SET still_edit = INPUT f'[*] Do you want to continue editing this
{member_type}? '
                                                f'(Press \'n\' -> No, \'y\' -> Yes): ').lower()
                                IF still_edit in ['y', 'n']:
                                    BREAK
                                ENDIF
                                PRINT f'[ERROR] Invalid option! Please enter again.'

                            IF still_edit == 'y':
                                CONTINUE
                            ELIF still_edit == 'n':
                                BREAK
```

```
                    ENDIF
                ELSE:
                    PRINT f'[ERROR] Invalid option! Please enter again.'
                BREAK
                ENDWHILE
            ENDIF
        ELSE:
            PRINT f'[ERROR] The ID {valid_id} does not exist in the database. Please enter
again.'
    CATCH ValueError:
        PRINT f'[ERROR] Invalid ID! Please enter a valid ID, or ensure the ID exists in the
database.'
END
```

Function to delete users

```
START
  FUNCTION delete_data():
    CALL load_data()
    GLOBAL user_type
    SET member_type = user_type.title()

    WHILE True:
      SET id = INPUT(f'[*] Enter the {member_type}\'s ID you want to delete (or type \'exit\'
to quit): ')

      IF id == 'exit':
        PRINT f'[*] Exiting the deleting process.\n'
        BREAK
      ENDIF

      TRY:
        SET max_id = max(int(id['id']) FOR id in user_data[member_type]) + 1
        SET valid_id = int(id)

        IF NOT 100 < valid_id < max_id:
          RAISE ValueError
        ENDIF

        FOR data in user_data[member_type]:
          IF valid_id == data['id']:
```

```
                user_data[member_type].remove(data)

                SET counter_id = 101
                FOR user in user_data[member_type]:
                   user['id'] = counter_id
                   counter_id += 1
                ENDFOR
                CALL save_data(user_database, user_data)
                PRINT f'[*] The {member_type} has been deleted successfully.'
            ENDFOR
         CATCH ValueError:
            PRINT f'[ERROR] Invalid ID! Please enter a valid ID, or ensure the ID exists in the
database.'
      ENDWHILE
END
```

Librarian Part

Interface for Librarian Account

```
START
  FUNCTION librarian_interface():
    WHILE True:
      CALL generate_title("Library Management System")
      SET option_list = ["Add Book", "Loan Process", "Edit Book", "View Books", "Delete Book", "Logout"]
      CALL generate_options(option_list)
      SET choice = INPUT("Enter your choice: ")
      IF choice == '1':
        CALL add_new_book()
      ELSE IF choice == '2':
        CALL loan_process()
      ELSE IF choice == '3':
        CALL edit_book()
      ELSE IF choice == '4':
        CALL view_books()
      ELSE IF choice == '5':
        CALL delete_book()
      ELSE IF choice == '6':
        PRINT "Exiting the Library Management System."
        BREAK
      ELSE
        PRINT "Invalid choice. Please try again."
        ENDIF
END
```

Function to load books data

```
START
FUNCTION load_book_data(file_name):
    TRY:
      OPENFILE file_name FOR READAS file
        RETURN json.load(file)
    CATCH FileNotFoundError:
      RETURN {"books": []}
END
```

Function to calculate overdue fee

```
START
 FUNCTION cal_overdue_fee(user):
    SET loan_data = CALL load_book_data(loans_file)
    SET current_time = datetime.now()
    TRY:
        SET total = 0
        FOR EACH item in loan_data:
        FOR item in loan_data[user]:
              SET due_date = datetime.strptime(item['due_date'], "%d-%m-%Y")
              SET overdue_days = (current_time - due_date).days
              SET fee = calculate_overdue_fee(overdue_days)
              SET item['overdue_fee'] = fee
              SET total += fee
          save_data(loans_file, loan_data)
          RETURN total
        CATCH KeyError:
          RETURN 0
```

Function to return overdue fee based on day

```
START
FUNCTION calculate_overdue_fee(days_passed):
    IF days_passed == 1:
        RETURN 2.00
    ELIF days_passed == 2:
        RETURN 3.00
    ELIF days_passed == 3:
        RETURN 4.00
    ELIF days_passed == 4:
        RETURN 5.00
    ELIF days_passed == 5:
        RETURN 6.00
    ELIF days_passed < 0:
        RETURN 0.00
    ELSE:
        RETURN 10.00
END
```

Function to get Due Date

```
START
FUNCTION get_due_date():
    SET date_str = INPUT("Enter the due date (dd-mm-yyyy): ")
    RETURN datetime.strptime(date_str, "%d-%m-%Y")
END
```

Function to add new books

START

FUNCTION add_new_book():

SET books_data = CALL load_book_data(books_file)

SET book_id = INPUT("Enter new book ID: ")

SET book_name = INPUT("Enter book name: ")

SET genre = INPUT("Enter genre: ")

SET author = INPUT("Enter author: ")

SET status = "Not Loaned"

SET new_book = {"book_ID": book_id, "book_name": book_name, "genre": genre, "author": author, "status": status}

APPEND new_book = books_data["books"]

CALL save_data(books_file, books_data)

PRINT f"New book '{book_name}' has been added to the library."

END

Function to edit book

START

FUNCTION edit_book():

SET books_data = CALL load_book_data(books_file)

SET book_id = INPUT("Enter the book ID to edit: ")

FOR EACH book IN books_data["books"]:

IF book["book_ID"] == book_id:

SET book["book_name"] = INPUT("Enter new book name: ")

SET book["genre"] = INPUT("Enter new genre: ")

SET book["author"] = INPUT("Enter new author: ")

SET book["status"] = INPUT("Enter new status: ")

```
        CALL save_data(books_file, books_data)
        PRINT f"Book '{book_id}' has been updated."
        RETURN
    PRINT f"Book '{book_id}' not found."
END
```

Function to view books

```
START
FUNCTION view_books():
    SET books = CALL load_book_data(books_file)
    PRINT TABLE HEADER ("BookID", "Name", "Genre", "Author", "Status")
    FOR EACH book IN books['books']
        PRINT book["book_ID"], book["book_name"], book["genre"], book["author"],
book["status"]
    ENDFOR
    SET exit = INPUT("Enter 'q' to 'exit'> ")
    IF exit.lower() == 'q':
        RETURN
    ENDIF
END
```

Function to delete books

```
START
FUNCTION delete_book():
    SET books_data = CALL load_book_data(books_file)
    SET book_id = INPUT("Enter the book ID to delete: ")
    FOR EACH book IN books_data["books"]:
```

```
        IF book["book_ID"] == book_id:
            REMOVE book FROM books_data["books"]
            CALL save_data(books_file, books_data)
            PRINT "Book '" + book_id + "' has been deleted."
            RETURN
         ENDIF
    ENDFOR
    PRINT f"Book '{book_id}' not found."
END
```

Function to make loan process

```
START
FUNCTION loan_process():
    SET loans = CALL load_book_data(loans_file)
    SET books = CALL load_book_data(books_file)
    WHILE True:
        TRY:
            SET username = INPUT("Enter the username: ")
            SET book_id = INPUT("Enter the book ID: ")
            BREAK
        CATCH ValueError
            PRINT 'Invalid Input. Please try again.'
    SET user_account = CALL load_data()
    IF username IN loans AND username IN (user['username'] FOR EACH user IN
user_account['Member']):
        IF LEN(loans[username]) >= 5:
            PRINT 'The user has already loaned 5 books.'
```

```
        ENDIF
            RETURN
    ELSE
        PRINT "The user doesn't exist."
        RETURN
    ENDIF

    SET overdue_fee = CALL cal_overdue_fee(username)
    IF overdue_fee > 0:
        PRINT 'The user has overdue fee and cannot proceed to the loan process!'
        RETURN
    ENDIF

    IF book_id IN (item['book_ID'] FOR EACH item IN books['books'])
        FOR EACH item IN books['books']
            IF item['status'] == 'Loaned' AND item['book_ID'] == book_id
                PRINT 'The book is already loaned.'
                RETURN
            IF item['status'] == 'Not Loaned' AND item['book_ID'] == book_id
                SET item['status'] = 'Loaned'
                CALL save_data(books_file, books)
                BREAK
    ELSE
        PRINT "The book id doesn't exist. Please try again."
        RETURN
    SET due_date = CALL get_due_date()
    SET time_difference = due_date - current_time
```

```
    IF time_difference.days >= 0
        PRINT "Time left until due date: " + time_difference.days + " days, " + hours + " hours, and " + minutes + " minutes."
    ELSE
        SET days_passed = ABS(time_difference.days)
        SET overdue_fee = CALL calculate_overdue_fee(days_passed)
        PRINT days_passed + " day(s) have passed since the due date."
        PRINT "Overdue fee: RM " + overdue_fee
    ENDIF

    PRINT "Username: " + username
    PRINT "Book ID: " + book_id

    FOR EACH book IN books['books']
        IF book_id == book['book_ID']
            SET book_info TO book
        ENDIF
    ENDFOR
    SET loan_info = {
            "book_ID": book_id,
            "book_name": book_info['book_name'],
            "genre": book_info['genre'],
            "author": book_info['author'],
            "status": 'Loaned',
            "loaned_date": current_time.strftime("%d-%m-%Y"),
            "due_date": due_date.strftime("%d-%m-%Y"),
            "overdue_fee": overdue_fee if time_difference.days < 0 else 0.00
    }
    IF username IN loans
        loans[username].append(loan_info)
```

```
    ELSE
        loans.update({username: [loan_info]})
    ENDIF
    CALL save_data(loans_file, loans)
    PRINT "Loan information has been updated."
END
```

Library Member Part

Library Member Interface

```
START

FUNCTION libmeminterface(user)

        CALL Generate_title:  "Welcome User [username]"

        SET  option_list=[1. Search books, 2. View loaned books, 3. Update profile, 4. Check_out book, 5. Logout]

        CALL Generate_options(option list)


   WHILE True

      TRY

         INPUT action from user

         IF action is between 1 and 5

            BREAK

         ELSE

            PRINT error message "Please Enter a number between 1 and 5"

         ENDIF

      CATCH error

         PRINT "You must enter a valid number!"

ENDWHILE


   IF action == 1

      CALL searchbooks(user)

   ELSE IF action == 2

      CALL viewloanedbooks(user)

   ELSE IF action == 3

      CALL update_profile(user)
```

```
    ELSE IF action == 4
        CALL book_checkout(user)
    ELSE
        CALL main()
END IF
END
```

Goback to the library member interface

```
START
FUNCTION goback(user)
    WHILE True
        INPUT backaction
        IF backaction == "Q"
            CALL libmeminterface(user)
        ELSE
            CONTINUE
        END IF
ENDWHILE
END
```

Function to view the user's loaned books

```
START
FUNCTION viewloanedbooks(user)
    PRINT "Here are your loaned books"

    OPEN loans_file FOR WRITE AS f:
    FOR each book in user's loaned books
```

```
        PRINT book details (ID, name, genre, author, loan date, due date, overdue fee)
    ENDFOR
    CALL goback(user)
END
```

Function to search the book for the user

```
START
FUNCTION searchbooks(user)
    OPEN books_file FOR WRITE AS f:
    PRINT search options: 1. Search by Book_ID, 2. Search by Status
    INPUT search choice

    IF search choice == 1
        INPUT book_id
        PRINT books with matching book_id
        CALL goback(user)

    ELSE IF search choice == 2
        INPUT status choice (1 for available books, 2 for loaned books)
        PRINT books based on status choice
        CALL goback(user)

    ELSE
        PRINT "Invalid option"
        CALL searchbooks(user)
ENDIF
END
```

Function to update the user's profile

```
START
FUNCTION update_profile(user)
   PRINT current user information

   WHILE True
      INPUT field to update (e.g., full_name, email, username, password)
      IF field is valid
         UPDATE field with new validated data
         SAVE updated data to user file

         INPUT if user wants to continue editing (y/n)
         IF no (n), EXIT loop and return to main interface
      ELSE
         PRINT "Invalid option"
      ENDIF
ENDWHILE
END
```

Function to check out the loaned books from the user side

```
START
FUNCTION book_checkout(user)
   LOAD books data
   LOAD loans data

   INPUT book_id to check out
```

```
    FOR each loaned book
        IF book_id matches
            REMOVE book from loans
            SAVE loans data
            PRINT "Book checked out successfully"
            MARK book status as "Not Loaned" in books data
            SAVE books data
            EXIT
ENDFOR

    IF no match found
        PRINT "Book id doesn't exist"
ENDIF
END
```

Program source code and explanation

Login Part

This function handles the user login by validating the entered email and password against stored user data.

It prompts for email and password, then checks if the entered credentials match any in the data dictionary. If a match is found, it displays a success message and returns the user's details; otherwise, it asks the user to retry.

```
def select_interface(user): # To display respective interfaces
    interface = user["role"]

    if interface == "admin":
        sys_admin(interface)
    if interface == 'super_admin':
        super_admin(interface)
    if interface == 'member':
        libmeminterface(user)
    if interface == 'librarian':
        librarian_interface()
```

Based on the user's role, this function directs them to the appropriate interface.

It checks the role in the user’s data (e.g., "admin", "super_admin", "member", "librarian") and calls the respective interface function.

```
def main(): # Main function of the system
    account_data = load_data()

    while True:
        user = login_users(account_data)
        if user:
            select_interface(user)
```

This function acts as the entry point of the system. It manages user login and interface selection.

It loads account data, continuously attempts to log in to the user, and once successful, passes the user data to select_interface() to display the corresponding interface options.

Admin Part

Interface for the admin

This function provides the interface for the admin using the function called options(). This function takes the options number from the admin user and opens the respective interface for the admin.

```
def sys_admin(role = 'admin'):
    picked_num = options(role)
    if picked_num == 1:
        member_manage(role)
    if picked_num == 2:
        librarian_manage(role)
    if picked_num == 3:
        print(f'\033[93m[*]\033[0m You have successfully logged out!\n')
        main()
```

Interface for super_admin

This function provides the interface for the super admin using the function called options(). This function takes the options number from the admin user and opens the respective interface for the super admin. Super admin account has the privilege to manage admin accounts within the system.

```
def super_admin(role = 'super_admin'):
    picked_num = options(role)
    if picked_num == 1:
        member_manage(role)
    if picked_num == 2:
        librarian_manage(role)
    if picked_num == 3:
        admin_manage()
    if picked_num == 4:
        print(f'\033[93m[*]\033[0m You have successfully logged out!\n')
        main()
```

Options for admin's interface

This function displays the interface for both admins and super admins based on their roles. It also calls the generate_title() and generate_options() functions to automatically generate the title and options. Then, it verifies that the user input is within the valid range. If value error occurs, the function generates error message to the user.

```
def options(role): #Main panel options
    generate_title('Welcome to Admin Panel')
    option_list = ['Member Account Management', 'Librarian Account Management', 'Logout']
    if role == 'super_admin':
        option_list.insert( __index: 2,  __object: 'Admin Account Management')
    generate_options(option_list)

    #To make sure the input is within valid range
    while True:
        picked_num = input(f'\033[93m[*]\033[0m Enter the number to manage the users: ')
        try:
            input_value = int(picked_num)

            if not 0 < input_value <= len(option_list):
                raise ValueError
            return input_value

        #To handle invalid inputs rather than integers
        except ValueError:
            print(f'\033[91m[ERROR] Invalid input! Please enter again.\033[0m')
            continue
```

Command Options

This function generates the options available within the admins' interfaces. The available options include add, view, search, edit, delete, options, and exit. Admin can manage user accounts using these commands.

```
def cmd_option(user_type): #Create the options for the admin's command
    options = f'''
    Available commands:
    - add: Add a new {user_type}
    - view: View {user_type} details
    - search: Search {user_type} information
    - edit: Edit {user_type} information
    - delete: delete a {user_type}
    - options: Show the available commnads
    - exit: Back to Main Menu
    '''
    print(options)
```

Save and Load Data

The save_data() function performs to save the data within the file in JSON data format. The load_data() checks the file already exists first before reading the data. If the file does not exist, it starts with empty data.

```
def save_data(file_name, data): #To store the data into the database
    with open(file_name, 'w') as f:
        json.dump(data, f, indent=4)

8 usages
def load_data(): #To load the data from the database
    global user_data
    try:
        if os.path.exists(user_database):
            with open(user_database, 'r') as f:
                user_data = json.load(f)
                return user_data
        else:
            user_data = {'Member': [], 'Librarian': [], 'Admin': []}
    except json.JSONDecodeError:
        user_data = {'Member': [], 'Librarian': [], 'Admin': []}
```

Generate Title and Options

These two functions aim to generate the title and options for all the interfaces. The generate title function needs the title name and prints the title with a specific format. The option generate function requires an option list and then it print all the options within the list with a number.

```
def generate_title(title): #Generate the title automatically
    print()
    print(f'{title.title()}')
    print(f'{"-" * 15}')

3 usages
def generate_options(option_list): #Generate the option automatically
    number = 1
    for i in option_list:
        print(f'{number}. {i.title()}')
        number += 1
    print()
```

Generate ID for Accounts

This is id automatic generation function for the users within the system. If there is no user in the database, it returns the default value of 101. If the user exists in the system, it returns the maximum number plus one.

```
def generate_id(): # To genearte id for users automatically
    load_data()
    global user_type
    member_type = user_type.title()

    #To return default id value if there is no user in the database
    if not user_data[member_type]:
        return 101

    #To return the maximun value + 1 if the database has data.
    max_id = max(int(user['id'])for user in user_data[member_type])
    return max_id + 1
```

Member management Interface

This is the function that generates the interface for library member management within the system.

It takes the command from the admin and performs a respective task. It returns the error message if the input is an invalid one.

```
def member_manage(role): #Member Management Panel
    print()
    generate_title('Member Account Management')
    print("\033[93m[*]\033[0m Use [options] to view the available commands.\n")

    while True:
        global user_type
        user_type = 'member'
        command = input(f'\033[91mAdmin\033[0m~# ').strip()
        if command == 'add':
            add_data()
        elif command == 'view':
            view_data()
        elif command == 'search':
            search_data()
        elif command == 'edit':
            edit_data()
        elif command == 'delete':
            delete_data()
        elif command == 'options':
            cmd_option(user_type)
        elif command == 'exit':
            print()
            if role == 'admin': #Check the admin role and provide the correct interface
                sys_admin()
            if role == 'super_admin':
                super_admin()
        else:
            print(f"\033[91m[ERROR] Unknown command. Type 'options' for a list of available commands.\033[0m")
```

Library Management Interface

This is the function that generates the interface for librarian account management in the system. It takes the command from the admin and performs the respective task. It returns the error message if the input is an invalid one.

```
def librarian_manage(role): #Librarian Management Panel
    print()
    generate_title('Librarian Account Management')
    print("\033[93m[*]\033[0m Use [options] to view the available commands.\n")

    while True:
        global user_type
        user_type = 'librarian'
        command = input(f'\033[91mAdmin\033[0m~# ').strip()
        if command == 'add':
            add_data()
        elif command == 'view':
            view_data()
        elif command == 'search':
            search_data()
        elif command == 'edit':
            edit_data()
        elif command == 'delete':
            delete_data()
        elif command == 'options':
            cmd_option(user_type)
        elif command == 'exit':
            print()
            if role == 'admin': #Check the admin role and provide the correct interface
                sys_admin()
            if role == 'super_admin':
                super_admin()
        else:
            print(f"\033[91m[ERROR] Unknown command. Type 'options' for a list of available commands.\033[0m")
```

Admin Management Interface

This is the function that generates the interface for admin account management in the system. It takes the command from the admin and performs the respective task. It returns the error message if the input is an invalid one.

```
def admin_manage(): #Admin Management Panel
    print()
    generate_title('Admin Account Management')
    print("\033[93m[*]\033[0m Use [options] to view the available commands.\n")

    while True:
        global user_type
        user_type = 'admin'
        command = input(f'\033[91mAdmin\033[0m~# ').strip()
        if command == 'add':
            add_data()
        elif command == 'view':
            view_data()
        elif command == 'search':
            search_data()
        elif command == 'edit':
            edit_data()
        elif command == 'delete':
            delete_data()
        elif command == 'options':
            cmd_option(user_type)
        elif command == 'exit':
            print()
            super_admin()
        else:
            print(f"\033[91m[ERROR] Unknown command. Type 'options' for a list of available commands.\033[0m")
```

Role Validation

This is the function to validate the role of users while adding to the system. First, it has a list containing all the roles. Then the function checks the user's input with all the roles. If the input is an invalid one, it returns the error message.

```
def role_validation(): #To validate the user role
    global user_type
    roles = ['member', 'librarian', 'admin']
    while True:
        role = input('Enter the role: ').lower().strip()

        if user_type.lower() == 'member' and role == roles[0]: #Check the role is member
                return role
        elif user_type.lower() == 'librarian' and role == roles[1]:  # Check the role is member
                return role
        elif user_type.lower() == 'admin' and role == roles[2]: #Check the role is member
                return role
        else:
            print(f'\033[91m[ERROR] Invalid role! Only "{user_type.lower()}" role is accepted.\033[0m')
            continue
```

User’s information validation

The following function is the validation function for the users’ information. It first creates a regex pattern for each field and matches the user information with that pattern. If the information does not match the criteria, it responds with the corresponding error message. Then, it also checks if there is any identical email, username, or full name within the system.

```
def validate_data(pattern_key): #To validate the user information
    #Patterns to validate the input from the users
    global user_type
    member_type = user_type.title()
    patterns = {
        'full_name': re.compile(r'^[a-zA-Z0-9][\w\s.-_]{0,18}$'),
        'email': re.compile(r'^[\w\.-]+@[\w\.-]+\.\w+$'),
        'username': re.compile(r'^[a-zA-Z0-9][\w.-]{1,13}$'),
        'password': re.compile(r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&]{1,5})[A-Za-z\d@$!%*?&]{8,}$')
    }
    pattern = patterns.get(pattern_key)

    if not pattern: #If there is no valid pattern in 'patterns' dictionary
        raise ValueError('Invalid Pattern Key!')

    while True:
        input_value = input(f'Enter the {pattern_key}: ').strip()
        match = pattern.fullmatch(input_value) #Match the input with the specific pattern

        if match:
            if pattern_key == 'email':
                if any(user['email'] == input_value for user in user_data[member_type]):
                    print(f'\033[91m[ERROR] This email is already registered. Please use a different email.\033[0m')
                    continue
            if pattern_key == 'full_name':
                if any(user['full_name'] == input_value for user in user_data[member_type]):
                    print(f'\033[91m[ERROR] This name is already used. Please choose a different first name.\033[0m')
                    continue
            if pattern_key == 'username':
                if any(user['username'] == input_value for user in user_data[member_type]):
                    print(f'\033[91m[ERROR] This username is already used. Please choose a different username.\033[0m')
```

```
                print(f'\033[91m[ERROR] This username is already used. Please choose a different username.\033[0m')
                continue
            return input_value

        else: #Raise the errors when the pattern is not matched with the input
            error_messages = {
                'full_name': 'Only alphabets, digits, and hyphens are allowed! Maximum length is 20 characters.',
                'email': 'Invalid email address format! Example: example@domain.com',
                'username': 'Invalid Username! Maximum length is 15 characters. Example: user_name123',
                'password': 'The password must be at least 8 characters long and include:\n- At least one uppercase letter\n- At least one l
            }
            print(f'\033[91m[ERROR] {error_messages.get(pattern_key, "Invalid input.")}\033[0m')
```

Function to show user's account information

This is the function to show user data in a specific format. It takes a data list and a counter to track the number of users.

```
def show_data(data_list, counter): #Function to dispaly the searched data
    print()
    print(f'\033[42m \033[30m\033[1m {'Id':<10} {'Full_name':<20} {'Email':<25} '
                    f'{'Username':<20} {'Password':<22} {'Role':<14} {'Registration_date':<20} \033[0m')
    for data in data_list:
        print(
            f'  {data.get("id"):<10} {data.get("full_name"):<20} {data.get("email"):<25} '
            f'{data.get("username"):<20} {data.get("password"):<22} {data.get("role"):<14} {data.get("registration_date"):<20}')
    print()
    if counter == 0:
        print(f'\n\033[93m[*] No data found!\033[0m')
    if counter > 0:
        print(f'\033[93m[*]\033[0m {counter} user{"" if counter == 1 or 0 else "s"} found!')
    print()
```

Viewing User Accounts

This function is to view the user's information using the show_data() function by providing the user data and the number of users.

```
def view_data(): #To view details information of the users
    load_data()
    global user_type
    member_type = user_type.title()
    show_data(user_data[member_type], len(user_data[member_type]))
```

Adding User Account

This function is to add the user to the system. First, it loads the data from the file using load_data() function. It takes the user's information in a dictionary format and validates it using validate_data() and role_validation() functions. Then, it saves the data in the file using save_data() function.

```
def add_data(): #To add user data into the database
    try:
        load_data()
        global user_type
        member_type = user_type.title() #Store the user for later usage

        data = {
            'id': generate_id(),
            'full_name': validate_data('full_name'),
            'email': validate_data('email'),
            'username': validate_data('username'),
            'password': validate_data('password'),
            'role': role_validation(),
            'registration_date': datetime.now().strftime('%d-%B-%Y')
        }

        #Add the member into the 'user_data' and store that in database using 'save_data' function.
        user_data[member_type].append(data)
        save_data(user_database, user_data)
        print(f'\033[92m[*] The {member_type} has been created successfully.\033[0m\n')
    except Exception as e:
        # Handle unexpected errors
        print(f'\033[91m[ERROR] An error occurred while adding the data: {e}\033[0m')
```

Searching User Account

This is the function to search the user's data in the system. First, it takes the field that the admin wants to search. After that, it takes the keyword from the admin and finds it within the data file using the search function from re module. At the same time, it also counts the number of users found in the system. Then, it uses show_data() function to show the result.

```
def search_data(): #To search the users using a specific keyword
    load_data()
    global user_type
    member_type = user_type.title()

    while True:
        option_list = ['id', 'full_name', 'email', 'username', 'exit']
        search_key = input(f'\033[93m\n[*]\033[0m Options: (id, full_name, email, username, \'exit\' to quit)\n'
                           f'Which field would you like to search?: ').lower()

        if search_key == 'exit':
            print(f'\033[93m[*] Exiting the searching process.\033[0m\n')
            break

        try:
            if not search_key in option_list: #To check the input is in option_list(line263)
                raise ValueError

            counter = 0
            search_value = input('Enter the keyword to search: ')
            data_list = []

            for data in user_data[member_type]: #To search the user data using the re module
                match_data = re.search(search_value, str(data[search_key]), flags=re.IGNORECASE)

                if match_data:
                    data_list.append(data)
                    counter += 1

            show_data(data_list, counter)
        except ValueError:
            print(f'\033[91m[ERROR] Invalid option! Please enter the valid option.\033[0m')
```

Editing User Account

This is the function to edit the user's information based on the field. First, it checks whether the input ID is valid or not. Then, it finds the ID within the data file and allows the admin to edit if the ID exists. After that, it takes the input from the user using validate_data() function while validating it. Finally, it saves the data in the file.

```
def edit_data(): #To edit the user data using 'id'
    view_data()
    global user_type
    member_type = user_type.title()

    while True:
        id = input(f'\033[93m[*]\033[0m Enter the {user_type}\'s ID you want to edit (or type \'exit\' to quit): ').lower()

        if id == 'exit':
            print(f'\033[93m[*]\033[0m Exiting the editing process.\n')
            break
        try:
            max_id = max(int(data['id']) for data in user_data[member_type]) + 1
            valid_id = int(id)  # Check whether the id is integer and within the valid range
            if not 100 < valid_id < max_id:
                raise ValueError("ID out of acceptable range.")
            for data in user_data[member_type]:  # loop the data inside database to compare with the id
                if valid_id == data['id']:
                    while True:  # Allow to choose the options to edit the user information
                        choice_to_edit = input(
                            f'\n\033[93m[*]\033[0m Options: (full_name, email, username, password)\n'
                            f'\033[93m[*]\033[0m Which field would you like to update?: ').lower()
                        option_list = ['full_name', 'email', 'username', 'password',]
                        if choice_to_edit in option_list:  # Check the input option is valid or not
                            edited_data = validate_data(choice_to_edit)
                            data[choice_to_edit] = edited_data
                            save_data(user_database, user_data)
                            print(f'\033[92m[*] The {member_type} has been edited successfully.\033[0m')

                            while True:
                                still_edit = input(f'\033[93m[*]\033[0m Do you want to continue editing this {member_type}? '
                                                   f'(Press \'n\' -> No, \'y\' -> Yes): ').lower()
                                if still_edit in ['y', 'n']:
                                    break
                                print(f'\033[91m[ERROR] Invalid option! Please enter again.\033[0m')
                            if still_edit == 'y':
                                continue
                            elif still_edit == 'n':
                                break

                        else:
                            print(f'\033[91m[ERROR] Invalid option! Please enter again.\033[0m')
                    break
            else:
                print(f'\033[91m[ERROR] The ID {valid_id} does not exist in the database. Please enter again.\033[0m')
        except ValueError:
            print(f'\033[91m[ERROR] Invalid ID! Please enter a valid ID, or ensure the ID exists in the database.\033[0m')
```

Deleting User Account

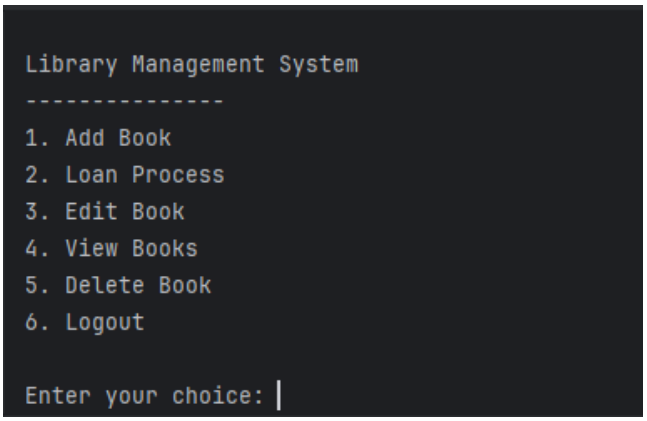
This is the function to delete the data of the user. It takes the user ID from the admin and validates it within the valid range. Then, it finds the ID within the data and deletes it. After that, it tries to rearrange the ID number to reserve and maintain the ID numbers.

```
def delete_data(): #Delete the user data from the database using 'id'
    load_data()
    global user_type
    member_type = user_type.title()

    while True:
        id = input(f'\033[93m[*]\033[0m Enter the {member_type}\'s ID you want to delete (or type \'exit\' to quit): ')

        if id == 'exit':
            print(f'\033[93m[*] Exiting the deleting process.\033[0m\n')
            break

        try: #Try to validate the id before remove from the database
            max_id = max(int(id['id']) for id in user_data[member_type]) + 1
            valid_id = int(id)
            if not 100 < valid_id < max_id: #To make sure the id is within the valid range
                raise ValueError
            for data in user_data[member_type]: #Find the id and remove from the database
                if valid_id == data['id']:
                    user_data[member_type].remove(data)

                    counter_id = 101 #Rearrange the id number after deleting the user
                    for user in user_data[member_type]:
                        user['id'] = counter_id
                        counter_id += 1
                    save_data(user_database, user_data)
                    print(f'\033[92m[*] The {member_type} has been deleted successfully.\033[0m')

        except ValueError:
            print(f'\033[91m[ERROR] Invalid ID! Please enter a valid ID, or ensure the ID exists in the database.\033[0m')
```

Librarian Part

Library Management System Interface

The provided code is part of a Library Management System, meticulously crafted to assist librarians in managing books and loans efficiently. Here's a comprehensive breakdown of the main functions:



Librarian Interface

The librarian_interface() function is pivotal, creating a user-friendly interface for librarians. It presents various options such as adding a book, processing loans, editing book details, viewing books, deleting books, and logging out. A loop ensures the menu is continuously displayed until the librarian opts to log out, ensuring seamless navigation and management.

```
449  def librarian_interface():  1 usage
451          generate_title("Library Management System")
452          option_list = ["Add Book", "Loan Process", "Edit Book", "View Books", "Dele
453          generate_options(option_list)
454
455          choice = input("Enter your choice: ")
456
457          if choice == '1':
458              add_new_book()
459          elif choice == '2':
460              loan_process()
461          elif choice == '3':
462              edit_book()
463          elif choice == '4':
464              view_books()
465          elif choice == '5':
466              delete_book()
467          elif choice == '6':
468              print("Exiting the Library Management System.")
469              break
470          else:
471              print("Invalid choice. Please try again.")
```

Loading Book Data

The load_book_data(file_name) function is designed to load book data from a specified file. If the file is non-existent, the function gracefully handles this by returning an empty dictionary with a "books" key, maintaining system stability.

```
472     def load_book_data(file_name):  9 usages
473         try:
474             with open(file_name, 'r') as file:
475                 return json.load(file)
476         except FileNotFoundError:
477             return {"books": []}
478
```

Calculating Overdue Fees

The cal_overdue_fee(user) function is essential for calculating the overdue fees for a user's borrowed books. It retrieves the loan data, computes the overdue days for each book, and determines the fee using the calculate_overdue_fee function. The total overdue fee is then meticulously saved back to the loan data file, ensuring accurate record-keeping.

```
479 def cal_overdue_fee(user):  2 usages
480     loan_data = load_book_data(loans_file)
481     current_time = datetime.now()
482
483
484     try:
485         total = 0
486         for item in loan_data[user]:
487             due_date = datetime.strptime(item['due_date'], format: "%d-%m-%Y")
488             overdue_days = (current_time - due_date).days
489             fee = calculate_overdue_fee(overdue_days)
490             item['overdue_fee'] = fee
491             total += fee
492         save_data(loans_file, loan_data)
493         return total
494     except KeyError:
495         return 0
```

Determining Overdue Fees

The calculate_overdue_fee(days_passed) function defines the overdue fee based on the number of days a book is overdue. The fee structure is progressive, increasing with the number of overdue days, which incentivizes timely returns and ensures fair billing.

```
def calculate_overdue_fee(days_passed):  2 usages
    if days_passed == 1:
        return 2.00
    elif days_passed == 2:
        return 3.00
    elif days_passed == 3:
        return 4.00
    elif days_passed == 4:
        return 5.00
    elif days_passed == 5:
        return 6.00
    elif days_passed < 0:
        return 0.00
    else:
        return 10.00
```

Getting Due Dates

The get_due_date() function prompts the user to enter a due date for a book and converts the input string into a datetime object. This ensures that due dates are accurately captured and processed, facilitating effective loan management.

```
def get_due_date():  1 usage
    date_str = input("Enter the due date (dd-mm-yyyy): ")
    return datetime.strptime(date_str,  format: "%d-%m-%Y")
```

Adding New Books

The add_new_book() function is a cornerstone for expanding the library's collection. It gathers new book details such as book ID, name, genre, and author, and integrates this information into the existing book data. The updated data is then securely saved back to the file, guaranteeing that the library's records are always up-to-date.

```
def add_new_book():  1 usage
    books_data = load_book_data(books_file)

    # New book details
    book_id = input("Enter new book ID: ")
    book_name = input("Enter book name: ")
    genre = input("Enter genre: ")
    author = input("Enter author: ")
    status = "Not Loaned"

    new_book = {
        "book_ID": book_id,
        "book_name": book_name,
        "genre": genre,
        "author": author,
        "status": status
    }

    # Add new book to books list
    books_data["books"].append(new_book)

    # Save updated books data
    save_data(books_file, books_data)
    print(f"New book '{book_name}' has been added to the library.")
```

```
Enter your choice: 1
Enter new book ID: B16
Enter book name: The Dark Knight Returns
Enter genre: Fiction,
Enter author: Frank Miller
New book 'The Dark Knight Returns' has been added to the library.
```

Editing, Viewing, and Deleting Books, and Loan Process

The latter part of the code extends the functionality of the Library Management System by introducing features to edit, view, and delete books, in addition to managing the loan process.

Editing Book Details

The edit_book() function empowers librarians to update the details of an existing book. By prompting for the book ID and updating fields such as name, genre, author, and status, this function ensures that book records can be kept current and accurate. If the book ID is not found, the librarian is notified, enhancing user experience and data integrity.

```
def edit_book():  1 usage
    books_data = load_book_data(books_file)
    book_id = input("Enter the book ID to edit: ")
    for book in books_data["books"]:
        if book["book_ID"] == book_id:
            book["book_name"] = input("Enter new book name: ")
            book["genre"] = input("Enter new genre: ")
            book["author"] = input("Enter new author: ")
            book["status"] = input("Enter new status: ")
            save_data(books_file, books_data)
            print(f"Book '{book_id}' has been updated.")
            return
    print(f"Book '{book_id}' not found.")
```

```
Enter your choice: 3
Enter the book ID to edit: B14
Enter new book name: Death Of-A Salesman
Enter new genre: Drama
Enter new author: Arthur Miller
Enter new status: Not Loaned
Book 'B14' has been updated.
```

Viewing Books

The view_books() function provides a comprehensive view of all books in the library. It prints the book ID, name, genre, author, and status in a well-formatted table, making it easy for librarians to review the library's inventory. The function allows the librarian to exit the view by entering 'q', offering flexibility in navigation.

```
def view_books():  1 usage
    books = load_book_data(books_file)

    print(f"\033[42m\033[30m{'BookID':<15} {'Name':<50} {'Genre':<20} {'Author':<20} {'Status':<15}\033[0m")
    for book in books['books']:
        print(f"{book['book_ID']:<15} {book['book_name']:<50} {book['genre']:<20} {book['author']:<20} {book['status']:<15}")
    exit = input("Enter 'q' to 'exit'> ")
    if exit.lower() == 'q':
        pass
```

```
Enter your choice: 4
BookID          Name                                    Genre             Author              Status
B01             1984                                    Fiction           George Orwell       Loaned
B02             Foundation                              Science Fiction   Isaac Asimov        Not Loaned
B03             Murder on the Orient Express            Mystery           Agatha Christie     Loaned
B04             Harry Potter and the Sorcerer's Stone   Fantasy           J.K. Rowling        Not Loaned
B05             Sapiens: A Brief History of Humankind   Non-fiction       Yuval Noah Harari   Not Loaned
B06             The Da Vinci Code                       Thriller          Dan Brown           Loaned
B07             The Pillars of the Earth                Historical Fiction Ken Follett        Not Loaned
B08             Pride and Prejudice                     Romance           Jane Austen         Not Loaned
B09             Steve Jobs                              Biography         Walter Isaacson     Loaned
B10             A Brief History of Time                 Science           Stephen Hawking     Not Loaned
B11             Twenty Thousand Leagues Under the Sea   Adventure         Jules Verne         Loaned
B12             The Road Not Taken                      Poetry            Robert Frost        Not Loaned
B13             The Shining                             Horror            Stephen King        Loaned
B14             Death of a Salesman                     Drama             Arthur Miller       Not Loaned
B15             How to Win Friends and Influence People Self-help         Dale Carnegie       Loaned
Enter 'q' to 'exit'>
```

Deleting Books

The delete_book() function facilitates the removal of books from the library's collection. By prompting for the book ID and deleting the corresponding record if found, this function ensures that outdated or unwanted books can be efficiently managed. If the book ID is not found, the librarian is informed, maintaining clarity and user satisfaction.

```python
# Function to delete a book from books.txt
def delete_book():  1 usage
    books_data = load_book_data(books_file)
    book_id = input("Enter the book ID to delete: ")
    for book in books_data["books"]:
        if book["book_ID"] == book_id:
            books_data["books"].remove(book)
            save_data(books_file, books_data)
            print(f"Book '{book_id}' has been deleted.")
            return
    print(f"Book '{book_id}' not found.")
```

```
Enter your choice: 5
Enter the book ID to delete: B15
Book 'B15' has been deleted.
```

Handling Loans

The loan_process() function is crucial for managing book loans. It verifies the user's existence and checks if they have already borrowed the maximum number of books (5). It also ensures that users with outstanding overdue fees are prevented from borrowing more books. The function checks the availability of the book and updates its status accordingly, ensuring accurate loan records.

```python
# Function to handle loan process
def loan_process():  1 usage
    # Load existing loan data
    loans = load_book_data(loans_file)
    books = load_book_data(books_file)

    while True:
        try:
            # Get username and book_id as input
            username = str(input("Enter the username: "))
            book_id = str(input("Enter the book ID: "))
            break
        except ValueError:
            print('Invalid Input. Please try again.')

    # Check the user has already loaned 5 books
    user_account = load_data()
    if username in loans and username in (user['username'] for user in user_account['Member']): #check the user exists in the a
        if len(loans[username]) >= 5:
            print('The user has already loaned 5 books.')
            return
    else:
        print("The user doesn't exist.")
        return
```

```
Library Management System
---------------
1. Add Book
2. Loan Process
3. Edit Book
4. View Books
5. Delete Book
6. Logout

Enter your choice: 2
Enter the username: Howard_223
Enter the book ID: B15
The user has overdue fee and cannot proceed to the loan process!
```

Additional Loan Process Details

The final segment of the code adds intricate details to the loan process, enhancing its robustness and user-friendliness.

```
# Check whether the user has overdue_fee or not
overdue_fee = cal_overdue_fee(username)
if overdue_fee > 0:
    print('The user has overdue fee and cannot proceed to the loan process!')
    return

# Check whether the book is loaned or not
if book_id in [item['book_ID'] for item in books['books']]:
    for item in books['books']:
        if item['status'] == 'Loaned' and item['book_ID'] == book_id:
            print('The book is already loaned.\n')
            return
        if item['status'] == 'Not Loaned' and item['book_ID'] == book_id:
            item['status'] = 'Loaned'
            save_data(books_file, books)
            break
else:
    print("The book id doesn't exists. Please try again.\n")
    return
```

Calculating Due Dates

The code calculates the time difference between the current date and the due date entered by the user. It displays the remaining time until the due date or the overdue fee if the due date has passed, providing clear information to the user.

```
# Get the due date from user input
due_date = get_due_date()

# Calculate the difference
time_difference = due_date - current_time

if time_difference.days >= 0:
    # If due date is in the future
    days = time_difference.days
    hours, remainder = divmod(time_difference.seconds, 3600)
    minutes, seconds = divmod(remainder, 60)
    print(f"Time left until due date: {days} days, {hours} hours, and {minutes} minutes.")
else:
    # If due date is in the past
    days_passed = abs(time_difference.days)
    overdue_fee = calculate_overdue_fee(days_passed)
    print(f"{days_passed} day(s) have passed since the due date.")
    print(f"Overdue fee: RM {overdue_fee:.2f}")
```

Updating Loan Information

The code meticulously updates loan information with details such as book ID, name, genre, author, loaned date, due date, and overdue fee. This updated loan data is then saved back to the file, ensuring that all transactions are recorded accurately and efficiently.

```
# Display user and book information
print(f"Username: {username}")
print(f"Book ID: {book_id}")

# Add or update loan information

for book in books['books']:
    if book_id == book['book_ID']:
        book_info = book

loan_info = {
    "book_ID": book_id,
    "book_name": book_info['book_name'],
    "genre": book_info['genre'],
    "author": book_info['author'],
    "status": 'Loaned',
    "loaned_date": current_time.strftime("%d-%m-%Y"),
    "due_date": due_date.strftime("%d-%m-%Y"),
    "overdue_fee": overdue_fee if time_difference.days < 0 else 0.00
}
```

```
# Update the loans dictionary
if username in loans: # To verify the user exists in loanedmembers.txt
    loans[username].append(loan_info)
else: # if the user doesn't exist, create a new object for user to store loaned books' information
    loans.update({username:[loan_info]})

# Save updated loan data
save_data(loans_file, loans)

print("Loan information has been updated.")
```

Library Member Part

```python
def libmeminterface(user):
    generate_title(f"Welcome User {user['username']}")
    option_list = ['Search books', 'View loaned books', 'Update profile', 'Check_out book', 'Logout']
    generate_options(option_list)

    while True:  # custom error message if they input strings
        try:
            action = int(input('Enter options: '))
            if 1 <= action <= 5:
                break
            else:
                print('Please Enter a number between 1 and 4!')
        except:
            print('You must enter a valid number!')

    if action == 1:
        searchbooks(user)
    elif action == 2:
        viewloanedbooks(user)
    elif action == 3:
        update_profile(user)  # need to create defs for the actions
    elif action == 4:
        book_checkout(user)
    else:
        main()
```

We start off the library member part by first defining the interface for members to see. Here, we generated the title to welcome the user by using generate_title and set option_list as a list. We used the generate options function to display the numbers matching with the options.

Then we continued with the Boolean statement to check whether the user input was correct or not. We made sure that the user can only input integers between 1 and 5. Error statement is added to make sure the numbers are valid.

After that the if loop is used to display the corresponding functions.

```
Welcome User Jamebob
---------------
1. Search Books
2. View Loaned Books
3. Update Profile
4. Check_Out Book
5. Logout

Enter options:
```

This is the interface output on the terminal with respective functions.

```python
def goback(user):  # going back to the main menu function
    while True:
        try:
            backaction = input("Enter 'q' to 'exit'> ").upper()
            if backaction == 'Q':
                libmeminterface(user)
            else:
                continue
        except ValueError:
            print("Enter a valid answer! ")
```

This goback function is a simple function that allows users to go back to the main interface. We put it after each function to give users an option to return.

```python
def viewloanedbooks(user):
    username = user['username']
    cal_overdue_fee(username)
    print('Here are your loaned books')
    print('----------------------')
    print(
        f"\033[42m\033[30m{'BookID':<15} {'Name':<45} {'Genre':<15} {'Author':<20} {'LoanDate':<15} {'DueDate':<15} {'OverDueFees':<15}\033
    allbooks = []
    with open(loans_file, 'r') as f:
        allbooks = json.load(f)
    for book in allbooks[user['username']]:  # change after i got user log in
        print(
            f"{book['book_ID']:<15} {book['book_name']:<45} {book['genre']:<15} {book['author']:<20} {book['loaned_date']:<15} {book['due_d
    goback(user)
```

Viewloanedbooks function allows the user to check the books they have loaned. In this function, we made sure to check that the username is the same as the user's so that it doesn't show other users' loaned books. Cal_overdue_fee(username) is called to check whether the user has overdue fees or not. We then opened the corresponding file as f and loaded it using JSON. For loop is used to display the books that were loaned by the user.

```
Enter options: 2
Here are your loaned books
----------------------
BookID          Name                                          Genre           Author               LoanDate        DueDate         OverDueFees
B01             1984                                          Fiction         George Orwell        1-09-2024       1-10-2024       10.0
Enter 'q' to 'exit'>
```

We can also see the function goback() being called at the bottom of the books.

```
def searchbooks(user):
    with open(books_file, 'r') as f:
        sbook = json.load(f)

    try:
        search = int(input(
            'WELCOME TO SEARCH MENU \n---------------------- \n1. Book_ID \n2. Status \nEnter the field you would like to search: '))

        if not 1 <= search <=2:
            raise ValueError

        print(f"\033[42m\033[30m{'BookID':<15} {'Name':<50} {'Genre':<20} {'Author':<15}\033[0m")
        for book in sbook['books']:
            print(f"{book['book_ID']:<15} {book['book_name']:<50} {book['genre']:<20} {book['author']}")

        if search == 1:
            bookid = input('Enter bookID: ').upper()
            print(f"\033[42m\033[30m{'BookID':<15} {'Name':<50} {'Genre':<15} {'Author':<15}\033[0m")
            for book in sbook['books']:
                if bookid == book['book_ID']:
                    print(f"{book['book_ID']:<15} {book['book_name']:<50} {book['genre']:<15} {book['author']:<15} ")
            goback(user)

         elif search == 2:
            status = int(input('1. Search available books \n2. Search loaned books \nEnter option:'))
            print(f"\033[42m\033[30m{'BookID':<15} {'Name':<50} {'Status':<15}\033[0m")
            with open(books_file, 'r') as f:
                avbook = json.load(f)
            if status == 2:
                for book in avbook['books']:
                    if book['status'] == 'Loaned':
                        print(f"{book['book_ID']:<15} {book['book_name']:<50} {book['status']:<15}")
                goback(user)
            elif status == 1:
                for book in sbook['books']:
                    if book['status'] == 'Not Loaned':
                        print(f" {book['book_ID']:<15} {book['book_name']:<50} {book['status']:<15}")
                goback(user)
            else:
                print('Enter a valid answer')
    except ValueError:
        print('Please enter a valid option.')
        searchbooks(user)
        goback(user)
```

For searchbook function, we first open the books_file to read it using JSON again. We put the entire code inside the try and except functions to make sure all of user's inputs are valid. The user can search in two categories; searching all the books in the library or searching only the available books by status. If the userinput is not 1 or 2, a value error will show up. We use the if loop for the search category. If the search is 1, then we will display all the books in the library. If it is 2, then we will display 2 more options to choose between. The first one being all the available books that are not loaned by any users and the second one being all the loaned books.

```
Enter options: 1
WELCOME TO SEARCH MENU
----------------------
1. Book_ID
2. Status
Enter the field you would like to search:
```

```
BookID          Name                                    Genre               Author
B01             1984                                    Fiction             George Orwell
B02             Foundation                              Science Fiction     Isaac Asimov
B03             Murder on the Orient Express            Mystery             Agatha Christie
B04             Harry Potter and the Sorcerer's Stone   Fantasy             J.K. Rowling
B05             Sapiens: A Brief History of Humankind   Non-fiction         Yuval Noah Harari
B06             The Da Vinci Code                       Thriller            Dan Brown
B07             The Pillars of the Earth                Historical Fiction  Ken Follett
B08             Pride and Prejudice                     Romance             Jane Austen
B09             Steve Jobs                              Biography           Walter Isaacson
B10             A Brief History of Time                 Science             Stephen Hawking
B11             Twenty Thousand Leagues Under the Sea   Adventure           Jules Verne
B12             The Road Not Taken                      Poetry              Robert Frost
B13             The Shining                             Horror              Stephen King
B14             Death of a Salesman                     Drama               Arthur Miller
B15             How to Win Friends and Influence People Self-help           Dale Carnegie
B16             Dark Horse                              Fiction             Ray
B17             The Sun                                 Fiction             Frank
Enter bookID: B02
BookID          Name                                    Genre           Author
B02             Foundation                              Science Fiction Isaac Asimov
Enter 'q' to 'exit'>
```

This is the first option that appears. We can enter the bookID to search for the book we want.

```
1. Search available books
2. Search loaned books
Enter option:1
BookID          Name                                    Status
 B03             Murder on the Orient Express           Not Loaned
 B04             Harry Potter and the Sorcerer's Stone  Not Loaned
 B05             Sapiens: A Brief History of Humankind  Not Loaned
 B06             The Da Vinci Code                      Not Loaned
 B07             The Pillars of the Earth               Not Loaned
 B08             Pride and Prejudice                    Not Loaned
 B10             A Brief History of Time                Not Loaned
 B16             Dark Horse                             Not Loaned
 B17             The Sun                                Not Loaned
Enter 'q' to 'exit'>
```

This is the second option with two more options within it. Since I chose number 1, it displays all the available books that haven't been loaned.

```
Enter option:2
BookID          Name                                        Status
B01             1984                                        Loaned
B02             Foundation                                  Loaned
B09             Steve Jobs                                  Loaned
B11             Twenty Thousand Leagues Under the Sea       Loaned
B12             The Road Not Taken                          Loaned
B13             The Shining                                 Loaned
B14             Death of a Salesman                         Loaned
B15             How to Win Friends and Influence People     Loaned
Enter 'q' to 'exit'>
```

The option 2 displays books that have already been loaned.

```python
def update_profile(user):
    load_data()
    global user_type
    user_type = 'Member'
    generate_title('User Information')
    restricted_field = ['role', 'registration_date']
    for key,value in user.items():
        if key in restricted_field:
            continue
        print(f"{key}: {value}")

    while True:
        try:
            option = input("Which field do you want to update (full_name, email, username, password): ")
            option_list = ['full_name', 'email', 'username', 'password', ]
            if option in option_list:
                for data in user_data['Member']:
                    if user['id'] == data['id']:
                        updated_data = validate_data(option)
                        data[option] = updated_data
                        new_user_info = data

                        if option == option_list[2]:
                            member_book = loans_file
                            with open(member_book, 'r') as f:
                                loaned_book_data = json.load(f)
```

```
                    if option == option_list[2]:
                        member_book = loans_file
                        with open(member_book, 'r') as f:
                            loaned_book_data = json.load(f)

                            if user['username'] in loaned_book_data:
                                loaned_book_data[updated_data] = loaned_book_data.pop(user['username'])
                        with open(member_book, 'w') as f:
                            json.dump(loaned_book_data, f, indent=4)

                    save_data(user_database, user_data)

                    exit = input("Do you want to still editing the data (Press 'n' -> No, 'y' -> Yes): ")
                    if exit.lower() == 'y':
                        continue
                    if exit.lower() == 'n':
                        print()
                        libmeminterface(new_user_info)
            else:
                print(f'\033[91m[ERROR] Invalid option! Please enter again.\033[0m')
        except ValueError:
            print("Please enter a valid number.")
```

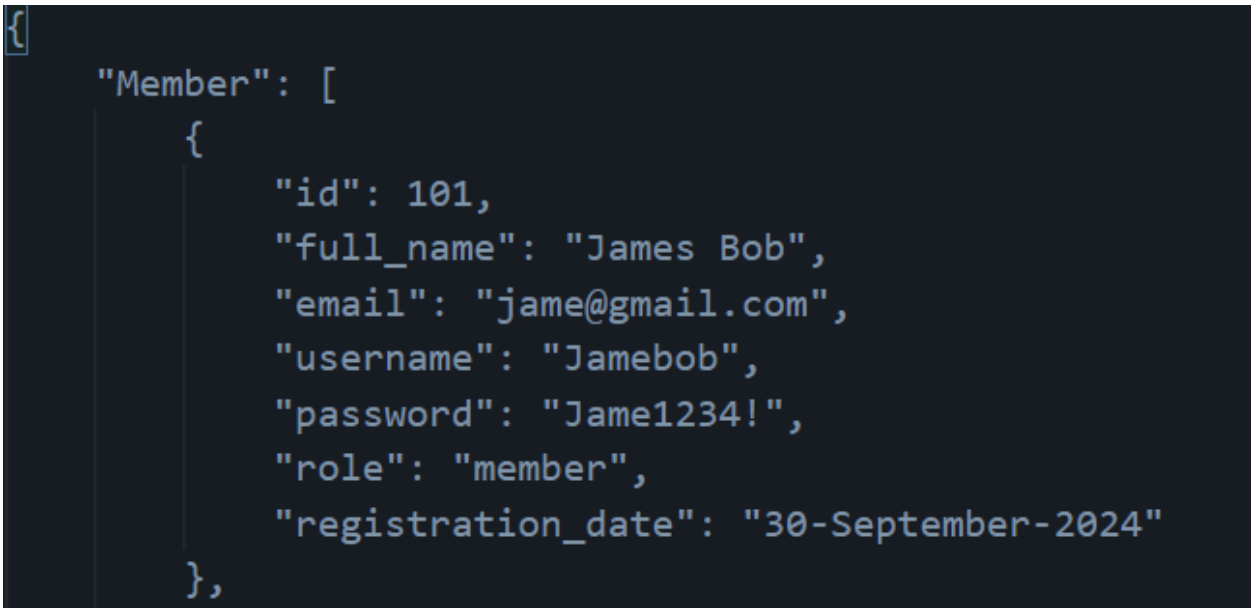
For the update_profile function, users can freely edit their user information except for their roles and registration date. We check on this by restricting the field of the role and registration_date by using restricted_field variable. The first if statement checks if the option is within the option_list. If that exists in the list, we match the current user's ID with the IDs in the user account file(in this case user_data['Member']). If that exists, we update the user info using validate_data() function validates the user's information. Then, we store that in updated_data. In the third if statement, if the user wants to change his username, we need to change the username in loanedmembers.txt. So, it opens the file and checks whether the user is in that loaned file. If the user exists, it replaces the old username with the new username. Then, it loads the updated data and it also calls save_data to change in the accounts.txt.

```
User Information
---------------
id: 101
full_name: Jame Bob
email: jame@gmail.com
username: Jamebob
password: Jame1234!
Which field do you want to update (full_name, email, username, password):
```

This is the output in the terminal for the update_profile function. You choose the field that you want to update.

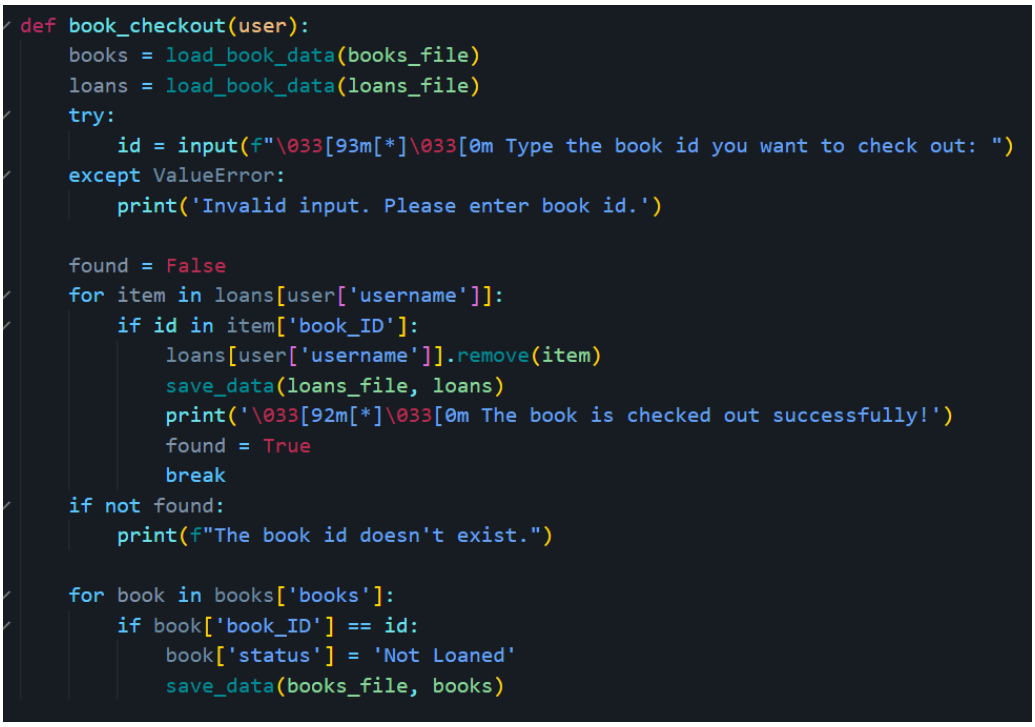
```
Which field do you want to update (full_name, email, username, password): full_name
Enter the full_name: James Bob
Do you want to still editing the data (Press 'n' -> No, 'y' -> Yes): n
```

After choosing the field and editing, it asks the user if they still want to edit the data.



```
{
    "Member": [
        {
            "id": 101,
            "full_name": "James Bob",
            "email": "jame@gmail.com",
            "username": "Jamebob",
            "password": "Jame1234!",
            "role": "member",
            "registration_date": "30-September-2024"
        },
```

When it's done, it saves in the account.txt file.



```
def book_checkout(user):
    books = load_book_data(books_file)
    loans = load_book_data(loans_file)
    try:
        id = input(f"\033[93m[*]\033[0m Type the book id you want to check out: ")
    except ValueError:
        print('Invalid input. Please enter book id.')

    found = False
    for item in loans[user['username']]:
        if id in item['book_ID']:
            loans[user['username']].remove(item)
            save_data(loans_file, loans)
            print('\033[92m[*]\033[0m The book is checked out successfully!')
            found = True
            break
    if not found:
        print(f"The book id doesn't exist.")

    for book in books['books']:
        if book['book_ID'] == id:
            book['status'] = 'Not Loaned'
            save_data(books_file, books)
```

The book_checkout function allows users to check out the book that they have. We first make sure the user is the correct user and load their loaned books. In for loop, if the bookID exists in his loaned books, we remove that book and save the file. And we display that "The book is

checked out successfully!" If it's not found, we will display "The book ID doesn't exist". In the last for loop, if the bookID status is "not Loaned" we will just save it.

```
Enter options: 4
[*] Type the book id you want to check out: B02
The book id doesn't exist.
```

To check out, I input the bookID that doesn't exist so it displays "The book id doesn't exist".

```
Enter options: 4
[*] Type the book id you want to check out: B01
[*] The book is checked out successfully!
```

```
        }
    ],
    "Jamebob": []
}
```

If the bookID exists, then it will check out successfully and that book will disappear from the user's loanedmembers.txt file.

Sample input/output and explanation

Login Part

This is the sample input and output for the login process. It returns the error output if the login user provides invalid information.

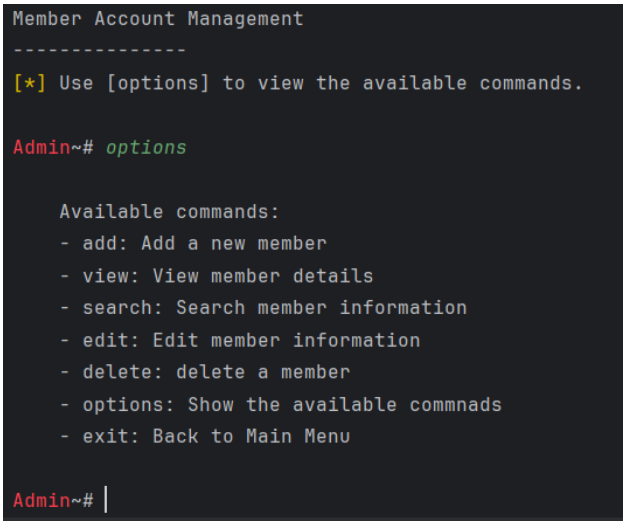
```
Welcome To Brickfields Kuala Lumpur Community Library
--------------
Enter your email address: admin@mail.com
Enter your password: $dmin@
[*] Invalid email or password, try again.

Enter your email address:
```

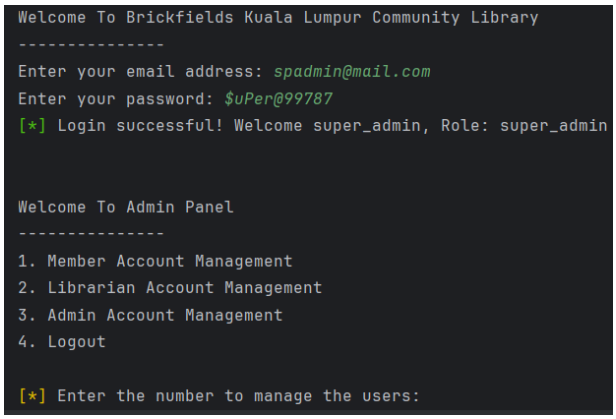
After successful login, the admin can see the user management panel.

```
Enter your email address: admin@mail.com
Enter your password: $dmin@99834
[*] Login successful! Welcome admin, Role: admin


Welcome To Admin Panel
--------------
1. Member Account Management
2. Librarian Account Management
3. Logout

[*] Enter the number to manage the users:
```

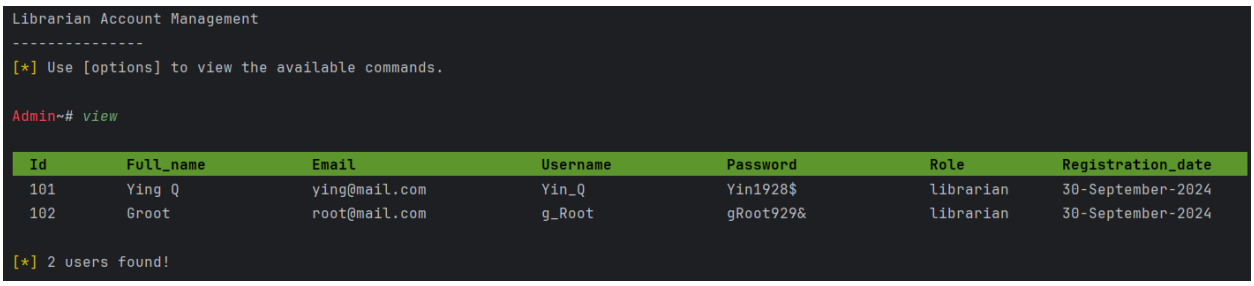
Admin Part

This is the sample input and output interfaces after the admin chooses the interface to manage. This interface is the same for librarians and member accounts management. There is also another interface for the super admin account. That is to provide admin accounts management within the system.





Below is the sample output of the view function within the admin part. This function shows the details information of the users, but it only displays the users based on the account management type. For example, it only displays librarian accounts here.



The following is the sample input for adding a librarian account to the system using the add function. This is where the validate_data() function works. It checks the input and response with an error message if the validation is not successful.

```
Admin~# add
Enter the full_name: David Jame
[ERROR] This name is already used. Please choose a different first name.
Enter the full_name: David_Jame
Enter the email: david.com
[ERROR] Invalid email address format! Example: example@domain.com
Enter the email: david@gmail.com
[ERROR] This email is already registered. Please use a different email.
Enter the email: davidj@gmail.com
Enter the username: Dav
Enter the password: david123
[ERROR] The password must be at least 8 characters long and include:
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character (@, $, !, %, *, ?, &)
Enter the password: D@vid123
Enter the role: librarian
[*] The Librarian has been created successfully.
```

This is the search function to find the librarians based on the field. Then it searches the user based on the keyword within the data and outputs the result.

```
Admin~# search

[*] Options: (id, full_name, email, username, 'exit' to quit)
Which field would you like to search?: email
Enter the keyword to search: dav

 Id   Full_name    Email              Username    Password   Role       Registration_date
 103  David Jame   david@gmail.com    david_jame  d@vid1234V librarian  31-October-2024
 104  David_Jame   davidj@gmail.com   Dav         D@vid123   librarian  31-October-2024

[*] 2 users found!
```

This is the editing function of the librarian account. In the second output, the full name is changed. When typing the field, the case is ignored for convenience.

```
Admin~# edit

 Id        Full_name          Email              Username     Password     Role        Registration_date
 101       Ying Q             ying@mail.com      Yin_Q        Yin1928$     librarian   30-September-2024
 102       Groot              root@mail.com      g_Root       gRoot929&    librarian   30-September-2024
 103       David Jame         david@gmail.com    david_jame   d@vid1234V   librarian   31-October-2024
 104       David_Jame         davidj@gmail.com   Dav          D@vid123     librarian   31-October-2024

[*] 4 users found!

[*] Enter the librarian's ID you want to edit (or type 'exit' to quit): B104
[ERROR] Invalid ID! Please enter a valid ID, or ensure the ID exists in the database.
[*] Enter the librarian's ID you want to edit (or type 'exit' to quit): 104

[*] Options: (full_name, email, username, password)
[*] Which field would you like to update?: Full_name
Enter the full_name: Frank
[*] The Librarian has been edited successfully.
[*] Do you want to continue editing this Librarian? (Press 'n' -> No, 'y' -> Yes): n
[*] Enter the librarian's ID you want to edit (or type 'exit' to quit): exit
[*] Exiting the editing process.
```

```
[*] The Librarian has been edited successfully.
[*] Do you want to continue editing this Librarian? (Press 'n' -> No, 'y' -> Yes): n
[*] Enter the librarian's ID you want to edit (or type 'exit' to quit): exit
[*] Exiting the editing process.

Admin~# view

 Id        Full_name          Email              Username     Password     Role        Registration_date
 101       Ying Q             ying@mail.com      Yin_Q        Yin1928$     librarian   30-September-2024
 102       Groot              root@mail.com      g_Root       gRoot929&    librarian   30-September-2024
 103       David Jame         david@gmail.com    david_jame   d@vid1234V   librarian   31-October-2024
 104       Frank              davidj@gmail.com   Dav          D@vid123     librarian   31-October-2024

[*] 4 users found!
```

This is the sample output of the delete function to delete a librarian account from the system.

```
Admin~# delete
[*] Enter the Librarian's ID you want to delete (or type 'exit' to quit): 104
[*] The Librarian has been deleted successfully.
[*] Enter the Librarian's ID you want to delete (or type 'exit' to quit): exit
[*] Exiting the deleting process.

Admin~# view

 Id        Full_name          Email              Username     Password     Role        Registration_date
 101       Ying Q             ying@mail.com      Yin_Q        Yin1928$     librarian   30-September-2024
 102       Groot              root@mail.com      g_Root       gRoot929&    librarian   30-September-2024
 103       David Jame         david@gmail.com    david_jame   d@vid1234V   librarian   31-October-2024

[*] 3 users found!
```

Librarian Part

This is the librarian interface to manage the book catalogues which includes adding, loaning, viewing, editing, and deleting books.

```
Library Management System
---------------
1. Add Book
2. Loan Process
3. Edit Book
4. View Books
5. Delete Book
6. Logout

Enter your choice: 4
```

This is the sample input and output to view all the book catalogues in the library system. This shows all the details information of the books such as genre, author and status.

```
Enter your choice: 4
BookID          Name                                    Genre              Author              Status
B01             1984                                    Fiction            George Orwell       Loaned
B02             Foundation                              Science Fiction    Isaac Asimov        Loaned
B03             Murder on the Orient Express            Mystery            Agatha Christie     Not Loaned
B04             Harry Potter and the Sorcerer's Stone   Fantasy            J.K. Rowling        Not Loaned
B05             Sapiens: A Brief History of Humankind   Non-fiction        Yuval Noah Harari   Not Loaned
B06             The Da Vinci Code                       Thriller           Dan Brown           Not Loaned
B07             The Pillars of the Earth                Historical Fiction Ken Follett         Not Loaned
B08             Pride and Prejudice                     Romance            Jane Austen         Not Loaned
B09             Steve Jobs                              Biography          Walter Isaacson     Loaned
B10             A Brief History of Time                 Science            Stephen Hawking     Not Loaned
B11             Twenty Thousand Leagues Under the Sea   Adventure          Jules Verne         Loaned
B12             The Road Not Taken                      Poetry             Robert Frost        Loaned
B13             The Shining                             Horror             Stephen King        Loaned
B14             Death of a Salesman                     Drama              Arthur Miller       Loaned
B15             How to Win Friends and Influence People Self-help          Dale Carnegie       Loaned
B16             Dark Horse                              Fiction            Ray                 Not Loaned
B17             The Sun                                 Fiction            Frank               Not Loaned
Enter 'q' to 'exit'>
```

This is the input and output when the librarian enters the user who has overdue fees into the system. Even if the user has one overdue fee, the system will not allow the user to loan another book.

```
Enter your choice: 2
Enter the username: Jamebob
Enter the book ID: B17
The user has overdue fee and cannot proceed to the loan process!
```

This is another sample input and output when the librarian enters the book that is already loaned. If the user wants to loan a book, the book needs to be not loaned to another user. The right one is the sample output if the librarian enters the user who doesn't exists in the library management system.

```
Enter your choice: 2
Enter the username: Howard_223
Enter the book ID: B15
The book is already loaned.
```

```
Enter your choice: 2
Enter the username: Omar
Enter the book ID: B17
The user doesn't exist.
```

This is how the system responds if the loan process is successful. To succeed in the loan process, the user must exist in the system. The book status should be not loaned and the user doesn't have any overdue fee. This is the example output for a successful loan process.

```
Enter your choice: 2
Enter the username: Howard_223
Enter the book ID: B17
Enter the due date (dd-mm-yyyy): 11-11-2024
Time left until due date: 10 days, 5 hours, and 52 minutes.
Username: Howard_223
Book ID: B17
Loan information has been updated.
```

This is how a book is added to the library through the system.

```
Enter your choice: 1
Enter new book ID: B18
Enter book name: Dark Night
Enter genre: Fiction
Enter author: George
New book 'Dark Night' has been added to the library.
```

This is the output of viewing books in the library. The new book is added to the system.

BookID	Name	Genre	Author	Status
B01	1984	Fiction	George Orwell	Loaned
B02	Foundation	Science Fiction	Isaac Asimov	Loaned
B03	Murder on the Orient Express	Mystery	Agatha Christie	Not Loaned
B04	Harry Potter and the Sorcerer's Stone	Fantasy	J.K. Rowling	Not Loaned
B05	Sapiens: A Brief History of Humankind	Non-fiction	Yuval Noah Harari	Not Loaned
B06	The Da Vinci Code	Thriller	Dan Brown	Not Loaned
B07	The Pillars of the Earth	Historical Fiction	Ken Follett	Not Loaned
B08	Pride and Prejudice	Romance	Jane Austen	Not Loaned
B09	Steve Jobs	Biography	Walter Isaacson	Loaned
B10	A Brief History of Time	Science	Stephen Hawking	Not Loaned
B11	Twenty Thousand Leagues Under the Sea	Adventure	Jules Verne	Loaned
B12	The Road Not Taken	Poetry	Robert Frost	Loaned
B13	The Shining	Horror	Stephen King	Loaned
B14	Death of a Salesman	Drama	Arthur Miller	Loaned
B15	How to Win Friends and Influence People	Self-help	Dale Carnegie	Loaned
B16	Dark Horse	Fiction	Ray	Not Loaned
B17	The Sun	Fiction	Frank	Loaned
B18	Dark Night	Fiction	George	Not Loaned

```
Enter 'q' to 'exit'>
```

This is the edit function to change the details of the book in the system.

```
Enter your choice: 3
Enter the book ID to edit: B18
Enter new book name: The King
Enter new genre: Potery
Enter new author: Alias
Enter new status: Not Loaned
Book 'B18' has been updated.
```

This is the result after editing the book details.

```
Enter your choice: 4
BookID  Name                                    Genre             Author             Status
B01     1984                                    Fiction           George Orwell      Loaned
B02     Foundation                              Science Fiction   Isaac Asimov       Loaned
B03     Murder on the Orient Express            Mystery           Agatha Christie    Not Loaned
B04     Harry Potter and the Sorcerer's Stone   Fantasy           J.K. Rowling       Not Loaned
B05     Sapiens: A Brief History of Humankind   Non-fiction       Yuval Noah Harari  Not Loaned
B06     The Da Vinci Code                       Thriller          Dan Brown          Not Loaned
B07     The Pillars of the Earth                Historical Fiction Ken Follett       Not Loaned
B08     Pride and Prejudice                     Romance           Jane Austen        Not Loaned
B09     Steve Jobs                              Biography         Walter Isaacson    Loaned
B10     A Brief History of Time                 Science           Stephen Hawking    Not Loaned
B11     Twenty Thousand Leagues Under the Sea   Adventure         Jules Verne        Loaned
B12     The Road Not Taken                      Poetry            Robert Frost       Loaned
B13     The Shining                             Horror            Stephen King       Loaned
B14     Death of a Salesman                     Drama             Arthur Miller      Loaned
B15     How to Win Friends and Influence People Self-help         Dale Carnegie      Loaned
B16     Dark Horse                              Fiction           Ray                Not Loaned
B17     The Sun                                 Fiction           Frank              Loaned
B18     The King                                Potery            Alias              Not Loaned
Enter 'q' to 'exit'>
```

Option number 5 is to delete the book from the library. This is the sample for deleting the book from the system. After deleting the book, the book no longer exist in the system.

```
Enter your choice: 5
Enter the book ID to delete: B18
Book 'B18' has been deleted.
```

```
Enter your choice: 4
BookID  Name                                    Genre             Author             Status
B01     1984                                    Fiction           George Orwell      Loaned
B02     Foundation                              Science Fiction   Isaac Asimov       Loaned
B03     Murder on the Orient Express            Mystery           Agatha Christie    Not Loaned
B04     Harry Potter and the Sorcerer's Stone   Fantasy           J.K. Rowling       Not Loaned
B05     Sapiens: A Brief History of Humankind   Non-fiction       Yuval Noah Harari  Not Loaned
B06     The Da Vinci Code                       Thriller          Dan Brown          Not Loaned
B07     The Pillars of the Earth                Historical Fiction Ken Follett       Not Loaned
B08     Pride and Prejudice                     Romance           Jane Austen        Not Loaned
B09     Steve Jobs                              Biography         Walter Isaacson    Loaned
B10     A Brief History of Time                 Science           Stephen Hawking    Not Loaned
B11     Twenty Thousand Leagues Under the Sea   Adventure         Jules Verne        Loaned
B12     The Road Not Taken                      Poetry            Robert Frost       Loaned
B13     The Shining                             Horror            Stephen King       Loaned
B14     Death of a Salesman                     Drama             Arthur Miller      Loaned
B15     How to Win Friends and Influence People Self-help         Dale Carnegie      Loaned
B16     Dark Horse                              Fiction           Ray                Not Loaned
B17     The Sun                                 Fiction           Frank              Loaned
Enter 'q' to 'exit'>
```

Library Member Part

After existing from the librarian account, we can go the library member's account. Then we can view the status of every book in the system in the user account, loaned books including overdue fee for each book, edit the user profile and check out the book. The following picture is the sample output of the library member interface.

```
Enter your choice: 6
Exiting the Library Management System.

Welcome To Brickfields Kuala Lumpur Community Library
---------------
Enter your email address: howard@gmail.com
Enter your password: Howard1!
[*] Login successful! Welcome Howard_223, Role: member


Welcome User Howard_223
---------------
1. Search Books
2. View Loaned Books
3. Update Profile
4. Check_Out Book
5. Logout

Enter options:
```

This is the sample input and output of viewing loaned books of the users where user can see details of the books and data related to the loan process including overdue fee for each book. Here, the user doesn't have any overdue fee, so that he can make a successful loan process. However, if his loaned books become five, he cannot make loan process anymore.

```
Enter options: 2
Here are your loaned books
----------------------
BookID          Name                                    Genre          Author           LoanDate      DueDate       OverDueFees
B15             How to Win Friends and Influence People Self-help      Dale Carnegie    27-10-2024    09-11-2024    0.0
B12             The Road Not Taken                      Poetry         Robert Frost     27-10-2024    09-11-2024    0.0
B14             Death of a Salesman                     Drama          Arthur Miller    29-10-2024    11-11-2024    0.0
B17             The Sun                                 Fiction        Frank            31-10-2024    11-11-2024    0.0
Enter 'q' to 'exit'>
```

This is the search function output of the user account. The user can search with 2 methods. The first one is searching books with ID and the second one is searching books with status.

```
Enter options: 1
WELCOME TO SEARCH MENU
----------------------
1. Book_ID
2. Status
Enter the field you would like to search: 1
BookID          Name                                    Genre               Author
B01             1984                                    Fiction             George Orwell
B02             Foundation                              Science Fiction     Isaac Asimov
B03             Murder on the Orient Express            Mystery             Agatha Christie
B04             Harry Potter and the Sorcerer's Stone   Fantasy             J.K. Rowling
B05             Sapiens: A Brief History of Humankind   Non-fiction         Yuval Noah Harari
B06             The Da Vinci Code                       Thriller            Dan Brown
B07             The Pillars of the Earth                Historical Fiction  Ken Follett
B08             Pride and Prejudice                     Romance             Jane Austen
B09             Steve Jobs                              Biography           Walter Isaacson
B10             A Brief History of Time                 Science             Stephen Hawking
B11             Twenty Thousand Leagues Under the Sea   Adventure           Jules Verne
B12             The Road Not Taken                      Poetry              Robert Frost
B13             The Shining                             Horror              Stephen King
B14             Death of a Salesman                     Drama               Arthur Miller
B15             How to Win Friends and Influence People Self-help           Dale Carnegie
B16             Dark Horse                              Fiction             Ray
B17             The Sun                                 Fiction             Frank
Enter bookID: B17
BookID          Name                                    Genre       Author
B17             The Sun                                 Fiction     Frank
Enter 'q' to 'exit'> q
```

This is the sample input and output for searching books with the status. The sample output provides all available books in the system.

```
1. Search available books
2. Search loaned books
Enter option:1
BookID          Name                                    Status
 B03             Murder on the Orient Express            Not Loaned
 B04             Harry Potter and the Sorcerer's Stone   Not Loaned
 B05             Sapiens: A Brief History of Humankind   Not Loaned
 B06             The Da Vinci Code                       Not Loaned
 B07             The Pillars of the Earth                Not Loaned
 B08             Pride and Prejudice                     Not Loaned
 B10             A Brief History of Time                 Not Loaned
 B16             Dark Horse                              Not Loaned
Enter 'q' to 'exit'>
```

The following is the sample input and output for updating the user profile from the user account. The user can change the full name, email, username and password.

```
Welcome User Howard_223
---------------
1. Search Books
2. View Loaned Books
3. Update Profile
4. Check_Out Book
5. Logout

Enter options: 3

User Information
---------------
id: 103
full_name: H0ward
email: howard@gmail.com
username: Howard_223
password: Howard1!
Which field do you want to update (full_name, email, username, password): username
Enter the username: Howard_111
Do you want to still editing the data (Press 'n' -> No, 'y' -> Yes): n


Welcome User Howard_111
---------------
1. Search Books
2. View Loaned Books
3. Update Profile
4. Check_Out Book
```

This is the sample input and output of checking out the book from the user account. The user can simply type the book ID that they want to check out.

```
Here are your loaned books
----------------------
BookID          Name                                    Genre          Author           LoanDate      DueDate       OverDueFees
B15             How to Win Friends and Influence People Self-help      Dale Carnegie    27-10-2024    09-11-2024    0.0
B12             The Road Not Taken                      Poetry         Robert Frost     27-10-2024    09-11-2024    0.0
B14             Death of a Salesman                     Drama          Arthur Miller    29-10-2024    11-11-2024    0.0
B17             The Sun                                 Fiction        Frank            31-10-2024    11-11-2024    0.0
Enter 'q' to 'exit'> q

Welcome User Howard_223
---------------
1. Search Books
2. View Loaned Books
3. Update Profile
4. Check_Out Book
5. Logout

Enter options: 4
[*] Type the book id you want to check out: B17
[*] The book is checked out successfully!
Enter 'q' to 'exit'> q
```

```
Enter options: 4
[*] Type the book id you want to check out: B17
[*] The book is checked out successfully!
Enter 'q' to 'exit'> q

Welcome User Howard_223
---------------
1. Search Books
2. View Loaned Books
3. Update Profile
4. Check_Out Book
5. Logout

Enter options: 2
Here are your loaned books
----------------------
BookID          Name                                    Genre          Author           LoanDate      DueDate       OverDueFees
B15             How to Win Friends and Influence People Self-help      Dale Carnegie    27-10-2024    09-11-2024    0.0
B12             The Road Not Taken                      Poetry         Robert Frost     27-10-2024    09-11-2024    0.0
B14             Death of a Salesman                     Drama          Arthur Miller    29-10-2024    11-11-2024    0.0
Enter 'q' to 'exit'>
```

Conclusion

In conclusion, the development of the Digital Library Management System for the Brickfields Kuala Lumpur Community Library enhances the system's efficiency and reliability significantly. The system minimizes waiting times and ensures greater accuracy by automating loan processes when compared to manual methods. Library members can update personal information, access the latest catalog updates, and check their loan books through the system, without needing direct assistance from librarians. Additionally, librarians can handle book records efficiently while reducing human errors, particularly in calculating overdue fees. The system includes admin accounts with the capabilities to add, view, search, edit, and delete all user accounts. The involvement of a super admin account further strengthens system security and reliability by overseeing all the user accounts. Overall, the implementation of this digital system provides a more user-friendly and dependable library environment, aligning with modern progress and advancement.

References

McGee, R. (1973). An analysis of manual circulation systems for academic libraries. *Journal of the American Society for Information Science*, *24*(3), 210–217. https://doi.org/10.1002/asi.4630240306

Drabenstott, J., Hayes, S., Belastock, T., Laucus, J., Cohen, D., Ross, G., McNally, B. J., Oltman, J. K., & Marquardt, S. (1989). Truth in Automating: Case studies in library automation. *Library Hi Tech*, *7*(1), 95–111. https://doi.org/10.1108/eb047752

Kong, X., & Xia, G. (2017). Analysis on the construction of Book Information Management System. *SciSpace - Paper*. https://typeset.io/papers/analysis-on-the-construction-of-book-information-management-95az1gmuo5

A, P., & Rathi, S. (2023). Designing a book recommendation system in library using collaborative filtering system. *Designing a Book Recommendation System in Library Using Collaborative Filtering System*. https://doi.org/10.1109/icccnt56998.2023.10307061

Putra, N. I. G. S. E., & Labasariyani, N. N. L. P. (2022). Automation identifier approach for library management information system. *Global Journal of Engineering and Technology Advances*, *12*(1), 110–119. https://doi.org/10.30574/gjeta.2022.12.1.0115